Order statistics solutions

09-E01 Comparisons for both extremes

Initialize min=max from the first element. The remaining ten form five pairs. Five internal comparisons determine pair extrema, then ten comparisons update the global extremes, totaling fifteen. The formula gives ceil(3 • 11/2)−2=17−2=15. The initial element already bounds both quantities and requires no comparison with itself.

09-E02 Random-select interval conversion

The pivot has global rank 5. Rank 7 lies right, whose global ranks are 6..10, so seek local rank 7−5=2. Rank 3 instead goes left with unchanged local rank 3. Rank 5 would return the pivot directly. Right conversion subtracts all known smaller elements and the pivot.

09-E03 Median-of-medians exclusion

Among g=ceil(n/5) medians, at least ceil(g/2) lie on either nonstrict side of x. Conservatively remove the short group and pivot group. At least ceil(g/2)−2 complete groups remain, each contributing at least three nonstrict witnesses. Thus each side has at least 3(ceil(g/2)−2)≥3n/10−6 records.

At n=50,g=10 this bound is 3(5−2)=9. A qualifying full group contributes **at least** three witnesses, not at most. Three-way recursion on a strict side excludes the other side and equality region, so duplicates do not break the bound.

09-E04 Local ranks with duplicates and a nonzero start

Since p=4, L=lt−p=1 and E=gt−lt+1=3. Equality occupies local ranks 2..4. Rank 1 continues in A[4..4] as rank 1; rank 4 returns 3; rank 6 continues in A[8..10] as rank 6−1−3=2.

lt,gt are absolute 0-based positions, whereas k is a local 1-based rank. Alternatively convert once to absolute target position p+k−1 and consistently retain it; then do not also subtract local ranks.

09-E05 Median-of-medians is not the exact median

Group medians are 4,8,12,14,15, whose lower median is 12. The full input is exactly 1..25, so there are 11 keys smaller, 1 equal, and 13 greater. Seek local rank 16−11−1=4 in the greater region. The actual median is rank 13, value 13, rather than 12.

Order statistics solutions (continued)

The pivot supplies a linear-cost balance guarantee, not a guaranteed immediate median. The first three groups each provide three values ≤ 12; the last three each provide three ≥ 12, giving nine explicit witnesses on each side. Pivot overlap does not invalidate separate branch bounds.

09-E06 Average larger side and expectation induction

The six sizes are 5,4,3,3,4,5. Their sum is 24 and average $4=3n/4-1/2$. Generally average is at most $3n/4$. Condition on each pivot and apply the shorter-state linear hypothesis to obtain $F\leq cn+3an/4$. Taking $a\geq 4c$ and covering bases yields $F\leq an$.

$E[T(S)]$ cannot generally become $T(E[S])$ for nonlinear T. First apply the expected-cost bound $\leq aS$ to each realized size, then use linearity. With duplicates, a fixed identity total order provides the same size upper bounds without changing actual key-only partitioning.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Order statistics solutions (continued)

Verification code appendix: selection_checks.py

```
"""Exact order-statistic oracles, median witnesses, and finite expectation sums."""
from functools import cache
from fractions import Fraction
from itertools import product, combinations_with_replacement
import random
import sys
from pathlib import Path
sys.path.insert(0,str(Path(__file__).resolve().parent))
from selection_partitions import partition3


GROUPS=[[1,2,4,17,25],[3,5,8,18,24],[6,9,12,19,23],[7,10,14,20,22],[11,13,15,16,21]]


def minmax(values):
    n=len(values)
    if not n:
        raise ValueError("nonempty input required")
    if n%2:
        lo=hi=values[0]
        start=1
        comparisons=0
    else:
        lo,hi=(values[0],values[1]) if values[0]<=values[1] else (values[1],values[0])
        start=2
        comparisons=1
    for i in range(start,n,2):
        a,b=values[i:i+2]
        small,large=(a,b) if a<=b else (b,a)
        comparisons+=1
        if small<lo:
            lo=small
        if large>hi:
            hi=large
        comparisons+=2
    return lo,hi,comparisons


def random_select(values,k,rng,prefix=()):
    if not 1<=k<=len(values):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Order statistics solutions (continued)

```
        raise ValueError("local1-index rank outside input")
    data=list(prefix)+list(values)
    p=len(prefix)
    r=len(data)-1
    while p<r:
        pivot=data[rng.randrange(p,r+1)]
        lt,i,gt=p,p,r
        while i<=gt:
            if data[i]<pivot:
                data[lt],data[i]=data[i],data[lt]
                lt+=1
                i+=1
            elif data[i]>pivot:
                data[i],data[gt]=data[gt],data[i]
                gt-=1
            else:
                i+=1
        left=lt-p
        equal=gt-lt+1
        if k<=left:
            r=lt-1
        elif k<=left+equal:
            return pivot
        else:
            k-=left+equal
            p=gt+1
    return data[p]


def deterministic_select(values,k,trace=None):
    """Readable list implementation: linear extra storage, not an in-place claim."""
    n=len(values)
    if not 1<=k<=n:
        raise ValueError("local1-index rank outside input")
    if n<=5:
        answer=sorted(values)[k-1]
        if trace is not None:
            trace.append((n,k,"base",answer))
        return answer
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Order statistics solutions (continued)

```
    groups=[sorted(values[i:i+5]) for i in range(0,n,5)]
    medians=[group[(len(group)-1)//2] for group in groups]
    pivot=deterministic_select(medians,(len(medians)+1)//2)
    left=[v for v in values if v<pivot]
    equal=[v for v in values if v==pivot]
    right=[v for v in values if v>pivot]
    assert min(n-len(left),n-len(right))>=Fraction(3*n,10)-6
    if trace is not None:
        trace.append((n,k,medians,pivot,(len(left),len(equal),len(right))))
    if k<=len(left):
        return deterministic_select(left,k,trace)
    if k<=len(left)+len(equal):
        return pivot
    return deterministic_select(right,k-len(left)-len(equal),trace)


@cache
def expected_scan_work(values,k):
    """Enumerate every equally probable pivot identity, retaining duplicate
multiplicities."""
    n=len(values)
    if n==1:
        return Fraction(1)
    future=Fraction(0)
    for pivot in values:
        left=tuple(v for v in values if v<pivot)
        equal=sum(v==pivot for v in values)
        right=tuple(v for v in values if v>pivot)
        if k<=len(left):
            future+=expected_scan_work(left,k)
        elif k>len(left)+equal:
            future+=expected_scan_work(right,k-len(left)-equal)
    return n+future/n


def main():
    arrays=0
    ranks=0
    rng=random.Random(20261024)
    for n in range(1,8):
        for values in product(range(3),repeat=n):
```

Order statistics solutions (continued)

```
            lo,hi,count=minmax(values)
            assert (lo,hi)==(min(values),max(values))
            assert count==(3*n+1)//2-2
            reference=sorted(values)
            for k in range(1,n+1):
                assert deterministic_select(values,k)==reference[k-1]
                assert random_select(values,k,rng,prefix=(999,-999))==reference[k-1]
                ranks+=1
            arrays+=1
    for n in [8,13,25,50,140,141,503,1001]:
        for _ in range(10):
            values=[rng.randrange(-n,n) for _ in range(n)]
            for k in {1,n,(n+1)//2,rng.randrange(1,n+1)}:
                assert deterministic_select(values,k)==sorted(values)[k-1]
    trace=[]
    answer=deterministic_select([v for group in GROUPS for v in group],16,trace)
    assert answer==16
    assert trace==[(25,16,[4,8,12,14,15],12,(11,1,13)),(13,4,[19,20,16],19,(6,1,6)),(6,4,
[15,16],15,(2,1,3)),(3,1,'base',16)]
    for n in range(1,501):
        assert Fraction(sum(max(r-1,n-r) for r in range(1,n+1)),n)<=Fraction(3*n,4)
    exact_states=0
    for n in range(1,10):
        for values in combinations_with_replacement(range(3),n):
            for k in range(1,n+1):
                assert expected_scan_work(values,k)<=4*n
                exact_states+=1
    for n in range(2,31):
        values=tuple(range(n))
        for k in range(1,n+1):
            assert expected_scan_work(values,k)<=4*n
            exact_states+=1
    assert partition3([8,3,5,3,9,1,3],3)==([1,3,3,3,9,5,8],1,3)
    print(f"PASS selection_arrays={arrays} all_ranks={ranks} exact_expected_states=
{exact_states} nonzero_start_rank=True n25_trace={trace} paired_minmax_count_formula=True")


if __name__=="__main__":
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Order statistics solutions (continued)

Verification code appendix: selection_partitions.py

```
"""Check three-way selection semantics and the all-equal Lomuto counterexample."""


def partition3(a, pivot):
    a = list(a)
    lt, i, gt = 0, 0, len(a) - 1
    while i <= gt:
        if a[i] < pivot:
            a[lt], a[i] = a[i], a[lt]
            lt += 1
            i += 1
        elif a[i] > pivot:
            a[i], a[gt] = a[gt], a[i]
            gt -= 1
        else:
            i += 1
    return a, lt, gt


def lomuto_comparisons(n):
    return n * (n - 1) // 2


def run():
    values = [3, 3, 1, 3, 2, 3]
    ordered, lt, gt = partition3(values, 3)
    assert ordered == [1, 2, 3, 3, 3, 3]
    assert (lt, gt) == (2, 5)
    target = 3
    assert lt <= target <= gt and ordered[target] == 3
    all_equal, lt2, gt2 = partition3([5] * 5, 5)
    assert all_equal == [5] * 5 and (lt2, gt2) == (0, 4)
    assert lomuto_comparisons(5) == 10
    assert lomuto_comparisons(10) == 45
    print("PASS partitions three_way=(2,5) all_equal=(0,4) lomuto_n5=10")
    return ordered, (lt, gt)


if __name__ == "__main__":
    run()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study